

b)

El equipo investiga cómo se debe aplicar conceptos de seguridad en las distintas etapas del desarrollo de software en un ciclo de vida:

Para integrar la seguridad de forma efectiva, las organizaciones aplican el Ciclo de Vida de Desarrollo de Software Seguro (SSDLC), fundamentado en marcos formales como el NIST SP 800-218 (Secure Software Development Framework) y el modelo OWASP SAMM. Este enfoque traslada la seguridad hacia las fases tempranas del proyecto (filosofía Shift-Left), evitando que la protección sea un parche tardío.

Fase de requisitos: Se definen los requerimientos no funcionales de seguridad, tales como el cifrado de datos en reposo y en tránsito, los controles de acceso basados en roles y el cumplimiento normativo. Asimismo, se formulan casos de abuso para anticipar cómo un atacante intentará romper las reglas del negocio.

Fase de diseño: Se realiza el modelado de amenazas para identificar debilidades estructurales antes de escribir código. Aquí rigen principios fundamentales como el privilegio mínimo, la defensa en profundidad y la separación de entornos.

Fase de codificación: Los desarrolladores aplican estándares de código seguro (como las directrices de OWASP Secure Coding Practices). Un principio crítico en esta etapa es la validación estricta de entradas en el lado del servidor. Dado que el entorno del cliente nunca debe considerarse de confianza, cualquier usuario puede manipular el DOM con las herramientas de desarrollo del navegador o enviar peticiones HTTP directas (usando cURL o Postman) para eludir los controles front-end. La seguridad en desarrollo exige que cada dato sea verificado, sanitizado y validado en el backend de forma obligatoria.

Fase de pruebas: Se implementan análisis automáticos en los flujos de integración continua, incluyendo análisis estático de código fuente, análisis dinámico de vulnerabilidades en tiempo de ejecución y análisis de dependencias de terceros para mitigar fallos en la cadena de suministro de software.

Fase de despliegue y mantenimiento: Se asegura el endurecimiento (hardening) de los servidores, la gestión centralizada de secretos sin exponer credenciales en el código, el monitoreo continuo de registros de auditoría y la planificación de respuesta ante incidentes para aplicar parches de forma oportuna.

Detalla mediante un cuadro cuáles son las buenas prácticas que se deben tener en cuenta.

Etapas del Ciclo de Vida	Buenas Prácticas Clave	Objetivo Principal
Requisitos	Identificación de requerimientos de seguridad, definición de casos de abuso y análisis de riesgos regulatorios.	Establecer los criterios de aceptación y defensas antes de iniciar el proyecto.
Diseño	Modelado de amenazas arquitectónicas (STRIDE), diseño de defensa en profundidad y principio de menor privilegio.	Prevenir fallas estructurales que resultan costosas de corregir más adelante.
Desarrollo	Validación obligatoria de datos en backend, codificación segura OWASP, sanitización de entradas y consultas preparadas.	Evitar que la seguridad dependa del navegador y neutralizar inyecciones de datos.
Pruebas	Ejecución de pruebas automáticas SAST, DAST, análisis SCA en dependencias y pruebas de penetración manuales.	Descubrir y remediar vulnerabilidades de código o librerías antes de producción.
Despliegue y Mantenimiento	Hardening de infraestructura, rotación de secretos, monitoreo centralizado de logs y gestión activa de parches.	Mantener la integridad del entorno operativo y responder con rapidez a incidentes.

Dar ejemplos de aplicaciones inseguras que no utilizan seguridad en el desarrollo de software. Por Ejemplo: “https://www.instagram.com/reels/DPNJZL2ESk0/”.

1. Aplicación con validación exclusiva en el cliente (caso del video de la consigna).

Una aplicación de comercio electrónico o registro que solo comprueba campos obligatorios mediante atributos HTML como "required" o scripts de JavaScript en el navegador. Al no contar con validaciones de negocio en el controlador o servicio del servidor, la base de datos almacena registros vacíos o inconsistentes.

2. Aplicación bancaria o administrativa vulnerable a SQL injection.

Un software desarrollado concatenando cadenas de texto sin validar para componer consultas hacia la base de datos (por ejemplo, "SELECT * FROM usuarios WHERE email = " + entradaUsuario + ""). Al no utilizar sentencias preparadas ni validaciones en la capa de persistencia, se podrían ingresar caracteres de escape para manipular la lógica de la consulta, logrando autenticarse sin contraseña o extrayendo la base de datos completa de los clientes.

3. Portal web sin encriptación de credenciales en el almacenamiento.

Un sistema donde las contraseñas de los usuarios se guardan en texto claro o utilizando algoritmos obsoletos y rotos como MD5 o SHA-1 sin salt. Ante una fuga de información o un acceso indebido al servidor de persistencia, los datos de los usuarios quedan expuestos inmediatamente.

4. Aplicación con fallas de Control de Acceso Roto (IDOR (Insecure Direct Object Reference) y **BOLA** (Broken Object Level Authorization)).

Un sistema que permite consultar facturas o datos médicos cambiando simplemente un parámetro en la dirección web (por ejemplo, cambiando la URL de "/factura?id=10" a "/factura?id=11") sin verificar si el usuario en sesión es el dueño legítimo de dicho recurso.

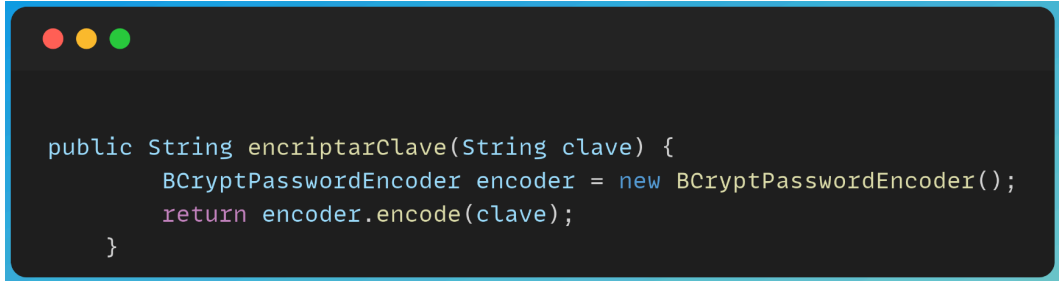
¿Qué significa que una clave de usuario se encuentre encriptada? Explicar y desarrollar los siguiente ejemplo el método de encriptación explicado en el siguiente video

Significa que el texto original de la contraseña se transforma en una cadena de caracteres incomprensible mediante una fórmula matemática antes de guardarse en la base de datos. Esto asegura que nadie pueda leer la contraseña real, ni siquiera los administradores del sistema o un atacante que logre acceder a la base de datos.

Los algoritmos de contraseñas (llamados hashes) están diseñados para que sea matemáticamente imposible hacer el proceso inverso (en encriptación si se puede hacer el proceso inverso). Es decir, no se puede descifrar el texto. A la hora de que el usuario inicie

sesión, se le aplica la misma función al texto ingresado y se compara la cadena resultante con el cifrado almacenado en la base de datos.

En el video se muestra el siguiente código:

A screenshot of a code editor window with a dark background and a light blue border. The window has three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Java and defines a method named `encriptarClave` that takes a `String` parameter `clave` and returns a `String`. The method creates a new `BCryptPasswordEncoder` instance and uses its `encode` method to encrypt the input key.

```
public String encriptarClave(String clave) {  
    BCryptPasswordEncoder encoder = new BCryptPasswordEncoder();  
    return encoder.encode(clave);  
}
```

que utiliza la funcionalidad `BCryptPasswordEncoder` de Spring Security para encriptar contraseñas. Spring Security es el módulo del ecosistema Spring para la gestión de autenticación (identificación de usuarios) y la autorización (permisos). Algunas de sus principales funcionalidades son control de acceso y redirección, autorización de roles, gestión de sesiones, y lo que nos importa ahora: encriptación de claves.

En el video se utiliza `BCryptPasswordEncoder`, que es una clase de Spring Security para realizar la encriptación de contraseñas. El método `encode(CharSequence rawPassword)` recibe como parámetro la contraseña ingresada por el usuario y le aplica la función matemática `bcrypt`.